

Question 1:What is a tuple in Python? Explain how tuples are created, accessed, and sliced. Why are tuples called immutable? Give one real-world situation where a tuple can be useful.

Answer :- 1. What is a Tuple in Python?

A tuple in Python is an ordered, immutable sequence of elements. It is one of the four built-in data types in Python used to store collections of data (alongside Lists, Sets, and Dictionaries).

Key characteristics of a tuple:

- **Ordered:** Elements maintain a defined order that will not change.
 - **Immutable:** Once created, elements cannot be added, removed, or modified.
 - **Allows Duplicates:** Since elements are indexed, tuples can store duplicate values.
 - **Heterogeneous:** A single tuple can contain elements of different data types (e.g., integers, strings, floats, booleans, and even nested objects).
-

2. Creation, Accessing, and Slicing of Tuples

A. Creating Tuples

Tuples are created using parentheses **()** or simply by separating values with commas (known as tuple packing).

- **Empty Tuple:**
- **Python**

```
empty_tuple = ()
```

-
-
- Tuple with Multiple Elements:
- Python

```
fruits = ("apple", "banana", "cherry", "date")
```

```
mixed_tuple = (10, "Hello", 3.14, True)
```

-
-
- Single-Element Tuple:
Note: A comma , is required after the element; otherwise, Python interprets it as a standard data type enclosed in parentheses.
- Python

```
single_item = ("apple",) # Correct: Type is tuple
```

```
not_a_tuple = ("apple") # Incorrect: Type is str
```

-
-
- Tuple Packing and Unpacking:
- Python

```
# Packing
```

```
coordinates = 10.5, 20.3
```

```
# Unpacking
```

```
x, y = coordinates
```

-
-

B. Accessing Elements in a Tuple

Tuple elements are accessed using indexing with square brackets `[]`. Indexing starts at **0**.

- Positive Indexing (Left-to-Right):
- Python

```
numbers = (10, 20, 30, 40)
```

```
print(numbers[0]) # Output: 10 (First element)
```

```
print(numbers[2]) # Output: 30 (Third element)
```

-
-

- Negative Indexing (Right-to-Left):
Negative indices count backwards starting from **-1**.
- Python

```
numbers = (10, 20, 30, 40)
```

```
print(numbers[-1]) # Output: 40 (Last element)
```

```
print(numbers[-2]) # Output: 30 (Second last element)
```

-
-

C. Slicing Tuples

Slicing allows retrieving a range of elements from a tuple. The standard syntax is:

`$$\text{tuple}[start : stop : step]$$`

- **start**: Index where the slice begins (inclusive).
- **stop**: Index where the slice ends (exclusive).
- **step**: Number of steps between items (default is **1**).

Examples of Slicing:

Python

```
data = (0, 10, 20, 30, 40, 50, 60, 70)
```

1. Basic Slicing (From index 1 up to index 4)

```
print(data[1:5]) # Output: (10, 20, 30, 40)
```

2. Slice from Start to a Specific Index

```
print(data[:3]) # Output: (0, 10, 20)
```

3. Slice from an Index to the End

```
print(data[4:]) # Output: (40, 50, 60, 70)
```

4. Slicing with Step Size

```
print(data[:2]) # Output: (0, 20, 40, 60)
```

5. Reversing a Tuple (Negative Step)

```
print(data[::-1]) # Output: (70, 60, 50, 40, 30, 20, 10, 0)
```

3. Why are Tuples Called Immutable?

Tuples are called immutable because their values or references cannot be modified after creation. Once assigned to memory, their structure remains constant.

Key Aspects of Immutability:

1. Item Reassignment Error: Attempting to modify an element at a specific index raises a **TypeError**.
2. Python

```
colors = ("red", "green", "blue")
```

```
colors[0] = "yellow" # TypeError: 'tuple' object does not support item assignment
```

- 3.
- 4.
5. No Addition or Removal Methods: Tuples do not have dynamic modification methods such as **.append()**, **.extend()**, **.insert()**, **.remove()**, or **.pop()**.

Advantages of Immutability:

- **Data Protection:** Ensures data remains safe from accidental modification during execution.
 - **Performance Optimization:** Python processes tuples faster than lists because of fixed memory allocation.
 - **Dictionary Keys:** Because tuples are immutable, they are hashable and can be used as keys in dictionaries (unlike lists).
-

4. Real-World Situation Where a Tuple is Useful

Scenario: Storing GPS Coordinates (Latitude & Longitude)

Example:

Python

```
# Location of a store represented as (Latitude, Longitude)
```

```
store_location = (28.6139, 77.2090) # New Delhi
```

Why a Tuple is Ideal Here:

1. **Data Integrity:** Geographic coordinates for a fixed landmark or office location should never change during program runtime. Using a tuple prevents accidental edits.
2. **Dictionary Key Compatibility:** If a delivery application maps coordinate locations to customer addresses or delivery personnel, tuples can serve directly as unique keys in a dictionary:
3. Python

```
delivery_routes = {
```

```
(28.6139, 77.2090): "Customer A - Central Delhi",  
  
(28.5355, 77.3910): "Customer B - Noida"  
  
}
```

4.

5. *(A list cannot be used for this purpose because lists are unhashable).*

Question 2: What is a dictionary? Explain how key-value pairs work in Python. Describe how to add, modify, delete, and access dictionary elements.

Answer :- A dictionary in Python is an unordered (or insertion-ordered in Python 3.7+), mutable collection of data stored in key-value pairs. It is one of Python's built-in mapping types and is created using curly braces `{ }` with keys and values separated by colons `:`.

Key Features of Dictionaries:

- **Key-Value Structure:** Every data item consists of a unique key mapped to a specific value.
- **Unique Keys:** Duplicate keys are not allowed. If a duplicate key is specified during creation or assignment, the last assigned value overwrites the previous one.
- **Immutable Keys:** Keys must be of an immutable (hashable) data type, such as strings, numbers, or tuples.
- **Mutable Values:** Values can be of any data type (integers, strings, lists, tuples, or even another dictionary) and can be modified or duplicated.

- **Fast Lookup:** Dictionaries use hash tables internally, allowing retrieval of values based on their keys in average $O(1)$ time complexity.
-

2. How Key-Value Pairs Work in Python

In a dictionary, data is organized as a collection of associations where a **Key** acts as an index or lookup name, and a **Value** represents the actual data associated with that key.

Syntax:

Python

```
dictionary_name = {  
    "key1": "value1",  
    "key2": "value2",  
    "key3": "value3"  
}
```

Mechanism (Hashing):

1. When a key-value pair is inserted, Python applies a hash function to the key to generate an integer hash value.
2. This hash value determines the exact memory location where the corresponding value is stored.

3. When accessing data using a key, Python hashes the key again to immediately jump to the stored location, bypassing the need to search through all elements sequentially.
-

3. Operations on Dictionary Elements

A. Accessing Dictionary Elements

Elements in a dictionary can be accessed using square brackets `[]` or the `.get()` method.

- Using Square Bracket Notation `[]`: Pass the key inside square brackets. If the key does not exist, Python raises a `KeyError`.
- Python

```
student = {"name": "Amit", "age": 21, "course": "B.Tech"}
```

```
print(student["name"]) # Output: Amit
```

```
print(student["course"]) # Output: B.Tech
```

-
-
- Using the `.get()` Method: Retrieves the value for a given key safely. If the key is not found, it returns `None` (or a specified default value) instead of raising an error.
- Python

```
print(student.get("age")) # Output: 21
```

```
print(student.get("grade", "N/A")) # Output: N/A (Key doesn't exist)
```

- -
-

B. Adding New Elements

You can add a new key-value pair by assigning a value to a new key using square brackets `[ ]` or by using the `.update()` method.

- Using Direct Assignment:
- Python

```
student = {"name": "Amit", "age": 21}
```

```
# Adding a new key "city"
```

```
student["city"] = "Delhi"
```

```
print(student)
```

```
# Output: {'name': 'Amit', 'age': 21, 'city': 'Delhi'}
```

-
-
- Using `.update()` Method:
- Python

```
student.update({"marks": 85})
```

```
print(student)
```

```
# Output: {'name': 'Amit', 'age': 21, 'city': 'Delhi', 'marks': 85}
```

- -
-

C. Modifying Existing Elements

Since dictionaries are mutable, you can update the value associated with an existing key by reassigning a new value to that key.

- Using Direct Reassignment:
- Python

```
student = {"name": "Amit", "age": 21, "city": "Delhi"}
```

```
# Modifying the 'age' key
```

```
student["age"] = 22
```

```
print(student)
```

```
# Output: {'name': 'Amit', 'age': 22, 'city': 'Delhi'}
```

-
-

- Using `.update()` Method:
- Python

```
student.update({"city": "Mumbai"})
```

```
print(student)
```

```
# Output: {'name': 'Amit', 'age': 22, 'city': 'Mumbai'}
```

-
-

D. Deleting Elements

Python provides multiple ways to remove items from a dictionary depending on the requirements:

1. **del** Statement: Removes an item by key or deletes the entire dictionary object.
2. Python

```
student = {"name": "Amit", "age": 22, "city": "Mumbai", "marks": 85}
```

```
del student["marks"] # Removes the 'marks' key-value pair
```

```
print(student)      # Output: {'name': 'Amit', 'age': 22, 'city': 'Mumbai'}
```

- 3.
- 4.
5. **.pop(key)** Method: Removes the specified key and returns its value.
6. Python

```
removed_city = student.pop("city")
```

```
print(removed_city) # Output: Mumbai
```

```
print(student)      # Output: {'name': 'Amit', 'age': 22}
```

7.

8.

9. **.popitem()** Method: Removes and returns the last inserted key-value pair as a tuple (**key**, **value**).

10. Python

```
last_item = student.popitem()
```

```
print(last_item)    # Output: ('age', 22)
```

11.

12.

13. **.clear()** Method: Removes all key-value pairs, leaving an empty dictionary.

14. Python

```
student.clear()
```

```
print(student)      # Output: {}
```

15.

16.

4. Summary of Common Dictionary Methods

Method	Description	Example
--------	-------------	---------

dict[key]	Accesses or creates/modifies a key-value pair.	student["age"] = 20
.get(key)	Returns value for key; returns default if missing.	student.get("name")
.keys()	Returns a view object containing all dictionary keys.	student.keys()
.values()	Returns a view object containing all values.	student.values()
.items()	Returns a view object containing key-value tuples.	student.items()
.pop(key)	Deletes specified key and returns its value.	student.pop("age")

Export to Sheets

Question 3: What is a set in Python? Explain why duplicate values are not stored in a set. Describe union, intersection, and difference with suitable examples.

Answer :- A set in Python is an unordered, unindexed collection of unique elements. Sets are mutable, but all individual elements inside a set must be immutable (hashable) types, such as numbers, strings, or tuples.

Python

Example of a set

```
my_set = {1, 2, 3, "apple"}
```

Why Duplicate Values Are Not Stored in a Set

Python sets do not allow duplicate values because of their internal hash table implementation:

1. **Hashability:** Every element added to a set is passed through Python's `hash()` function to generate a unique hash value.
2. **Key Uniqueness:** The hash table uses these hash values to decide where to store the element. If an element with the same hash value and equality (`==`) already exists in the set, Python ignores or overwrites the duplicate entry.
3. **Constant Time Lookup:** Hashing ensures that membership checks (e.g., `item in my_set`) happen efficiently in $O(1)$ average time complexity.

Set Operations: Union, Intersection, and Difference

1. Union ($\cup$)

Combines all unique elements from both sets.

- **Operator:** |
- **Method:** .union()

Python

```
set_a = {1, 2, 3}
```

```
set_b = {3, 4, 5}
```

Using operator

```
result = set_a | set_b
```

```
print(result) # Output: {1, 2, 3, 4, 5}
```

Using method

```
result_method = set_a.union(set_b)
```

2. Intersection ($\cap$)

Returns only the elements that are common to both sets.

- **Operator:** &
- **Method:** `.intersection()`

Python

```
set_a = {1, 2, 3}
```

```
set_b = {3, 4, 5}
```

Using operator

```
result = set_a & set_b
```

```
print(result) # Output: {3}
```

Using method

```
result_method = set_a.intersection(set_b)
```

3. Difference (\$-\$)

Returns elements present in the first set but **not** in the second set.

- **Operator:** -
- **Method:** `.difference()`

Python

```
set_a = {1, 2, 3}
```

```
set_b = {3, 4, 5}
```

```
# Elements in set_a but not in set_b
```

```
diff_a = set_a - set_b
```

```
print(diff_a) # Output: {1, 2}
```

```
# Elements in set_b but not in set_a
```

```
diff_b = set_b.difference(set_a)
```

```
print(diff_b) # Output: {4,
```

Question 4:What are conditional statements in Python? Explain if, if-else, if-elif-else, and nested if statements with suitable examples

Answer :- Conditional statements in Python are decision-making statements used to execute specific blocks of code based on whether a given condition evaluates to **True** or **False**. They allow a program to control its flow of execution based on different inputs or states.

Types of Conditional Statements

1. **if Statement**

The **if** statement executes a block of code only if the specified condition is **True**. If the condition is **False**, the code block is skipped.

Syntax:

Python

if condition:

 # Code executes if condition is True

Example:

Python

age = 20

if age >= 18:

 print("You are eligible to vote.")

2. **if-else** Statement

The **if-else** statement provides an alternative path. If the condition is **True**, the **if** block executes. Otherwise, the **else** block executes.

Syntax:

Python

if condition:

 # Code executes if condition is True

else:

 # Code executes if condition is False

Example:

Python

num = 7

if num % 2 == 0:

 print("Even number")

else:

 print("Odd number")

3. if-elif-else Statement

The `if-elif-else` statement is used when you need to evaluate multiple conditions in sequence. Python checks each condition from top to bottom; once it finds a `True` condition, it executes that block and skips the rest. If none of the conditions are `True`, the `else` block executes.

Syntax:

Python

```
if condition1:
```

```
    # Code executes if condition1 is True
```

```
elif condition2:
```

```
    # Code executes if condition2 is True
```

```
else:
```

```
    # Code executes if all conditions are False
```

Example:

Python

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade: A+")
```

```
elif marks >= 75:
```

```
    print("Grade: A")
```

```
elif marks >= 50:
```

```
    print("Grade: B")
```

```
else:
```

```
    print("Grade: Fail")
```

4. Nested **if** Statement

A nested **if** statement contains an **if** or **if-else** block inside another **if** or **else** block. It is useful when a secondary decision depends on the outcome of a primary decision.

Syntax:

Python

```
if outer_condition:
```

```
    if inner_condition:
```

```
        # Code executes if both conditions are True
```

```
    else:
```

```
        # Code executes if outer is True, but inner is False
```

else:

Code executes if outer condition is False

Example:

Python

num = 15

if num >= 0:

if num == 0:

print("Zero")

else:

print("Positive number")

else:

print("Negative number")

Question 5: Explain the difference between for and while loops. Also explain the purpose of break, continue, pass, and the else clause with loops.

Answer :- Difference Between **for** and **while** Loops

In Python, loops are used to execute a block of code repeatedly. The primary difference between a **for** loop and a **while** loop lies in how the iteration is controlled:

Feature	for Loop	while Loop
Purpose	Used when the number of iterations is known or fixed in advance.	Used when the number of iterations is unknown and depends on a condition.
Control Mechanism	Iterates directly over a sequence (list, tuple, string, range, etc.).	Continues running as long as a specified condition evaluates to True .
Risk of Infinite Loop	Minimal (stops automatically when the sequence ends).	High (requires manually updating condition variables inside the loop).

Code Examples:

1. **for** Loop Example:

Python

Iterates through a fixed range of numbers

```
for i in range(1, 4):
```

```
    print(i)
```

Output: 1, 2, 3

2. **while** Loop Example:

Python

Continues as long as the condition (count <= 3) is True

```
count = 1
```

```
while count <= 3:
```

```
    print(count)
```

```
    count += 1
```

Output: 1, 2, 3

Loop Control Statements and the **else** Clause

1. **break** Statement

Terminates the loop immediately when executed, skipping any remaining iterations and exiting the loop.

Python

```
for num in range(1, 10):
```

```
    if num == 5:
```

```
        break # Stops and exits the loop when num is 5
```

```
    print(num)
```

```
# Output: 1, 2, 3, 4
```

2. **continue** Statement

Skips the rest of the code inside the loop for the current iteration and jumps directly to the next iteration.

Python

```
for num in range(1, 6):
```

```
    if num == 3:
```

```
        continue # Skips printing 3 and moves to the next number
```

```
    print(num)
```

```
# Output: 1, 2, 4, 5
```

3. **pass** Statement

Acts as a placeholder or null statement. It does nothing when executed and is used to avoid syntax errors when writing empty loop bodies.

Python

```
for num in range(1, 5):  
  
    if num == 2:  
  
        pass # Placeholder for future code  
  
    print(num)
```

Output: 1, 2, 3, 4

4. **else** Clause with Loops

The **else** block attached to a loop executes only if the loop finishes naturally without being interrupted by a **break** statement.

Python

Example A: Loop completes naturally (else executes)

```
for i in range(3):  
  
    print(i)
```

else:

print("Loop completed normally.")

Example B: Loop interrupted by break (else is skipped)

for i in range(3):

if i == 1:

break

else:

print("This will not print.")

Question 6: Create the following tuple:

marks = (78, 85, 90, 67, 88) Write a Python program to:

1. Display the first and last values.
2. Display values from index 1 to 3.
3. Find the highest and lowest marks.
4. Convert the tuple into a list.
5. Add 95 to the list.
6. Convert the updated list back into a tuple.

Answer :- Python

Create the tuple

```
marks = (78, 85, 90, 67, 88)
```

1. Display the first and last values

```
first_val = marks[0]
```

```
last_val = marks[-1]
```

```
print("First value:", first_val)
```

```
print("Last value:", last_val)
```

2. Display values from index 1 to 3 (inclusive of index 3)

```
sliced_marks = marks[1:4]
```

```
print("Values from index 1 to 3:", sliced_marks)
```

3. Find the highest and lowest marks

```
highest_mark = max(marks)
```

```
lowest_mark = min(marks)
```

```
print("Highest mark:", highest_mark)
```

```
print("Lowest mark:", lowest_mark)
```

```
# 4. Convert the tuple into a list
```

```
marks_list = list(marks)
```

```
print("Converted to list:", marks_list)
```

```
# 5. Add 95 to the list
```

```
marks_list.append(95)
```

```
print("List after adding 95:", marks_list)
```

```
# 6. Convert the updated list back into a tuple
```

```
updated_marks_tuple = tuple(marks_list)
```

```
print("Updated tuple:", updated_marks_tuple)
```

Program Output

Plaintext

First value: 78

Last value: 88

Values from index 1 to 3: (85, 90, 67)

Highest mark: 90

Lowest mark: 67

Converted to list: [78, 85, 90, 67, 88]

List after adding 95: [78, 85, 90, 67, 88, 95]

Updated tuple: (78, 85, 90, 67, 88, 95)

Question 7: Create a dictionary to store:

- Student name
- Roll number
- Course
- Marks Write a Python program to:

1. Display all student details.
2. Add the student's city.
3. Update the marks.
4. Display all keys and values.
5. Iterate through the dictionary and print each key with its value .

Answer :- Python

Create dictionary with initial student details

```
student = {  
    "name": "Rahul Sharma",  
    "roll_no": 101,  
    "course": "B.Tech Computer Science",  
    "marks": 88  
}
```

1. Display all student details

```
print("1. Initial Student Details:")  
  
print(student)  
  
print("-" * 40)
```

2. Add the student's city

```
student["city"] = "Bhopal"  
  
print("2. After adding City:")  
  
print(student)
```

```
print("-" * 40)
```

```
# 3. Update the marks
```

```
student["marks"] = 95
```

```
print("3. After updating Marks:")
```

```
print(student)
```

```
print("-" * 40)
```

```
# 4. Display all keys and values separately
```

```
print("4. Keys and Values:")
```

```
print("Keys:", list(student.keys()))
```

```
print("Values:", list(student.values()))
```

```
print("-" * 40)
```

```
# 5. Iterate through the dictionary and print each key with its value
```

```
print("5. Iterating through Dictionary:")
```

```
for key, value in student.items():
```

```
    print(f"{key}: {value}")
```

Program Output

Plaintext

1. Initial Student Details:

```
{'name': 'Rahul Sharma', 'roll_no': 101, 'course': 'B.Tech Computer Science', 'marks': 88}
```

2. After adding City:

```
{'name': 'Rahul Sharma', 'roll_no': 101, 'course': 'B.Tech Computer Science', 'marks': 88, 'city': 'Bhopal'}
```

3. After updating Marks:

```
{'name': 'Rahul Sharma', 'roll_no': 101, 'course': 'B.Tech Computer Science', 'marks': 95, 'city': 'Bhopal'}
```

4. Keys and Values:

Keys: ['name', 'roll_no', 'course', 'marks', 'city']

Values: ['Rahul Sharma', 101, 'B.Tech Computer Science', 95, 'Bhopal']

5. Iterating through Dictionary:

name: Rahul Sharma

roll_no: 101

course: B.Tech Computer Science

marks: 95

city: Bhopal

Question 8: Two classes have registered for different activities.

sports = {"Rahul", "Anil", "Priya", "Kiran"}

music = {"Priya", "Kiran", "Meena", "Arun"}

Write a Python program to find:

1. Students who joined either activity.
2. Students who joined both activities.
3. Students who joined only sports.
4. Students who joined only music.

Answer :- Python

```
# Define the sets of students
```

```
sports = {"Rahul", "Anil", "Priya", "Kiran"}
```

```
music = {"Priya", "Kiran", "Meena", "Arun"}
```

```
# 1. Students who joined either activity (Union)
```

```
either_activity = sports | music
```

```
print("1. Students who joined either activity:", either_activity)
```

```
# 2. Students who joined both activities (Intersection)
```

```
both_activities = sports & music
```

```
print("2. Students who joined both activities:", both_activities)
```

```
# 3. Students who joined only sports (Difference: sports - music)
```

```
only_sports = sports - music
```

```
print("3. Students who joined only sports:", only_sports)
```

```
# 4. Students who joined only music (Difference: music - sports)
```

```
only_music = music - sports
```

```
print("4. Students who joined only music:", only_music)
```

Program Output

Plaintext

1. Students who joined either activity: {'Anil', 'Priya', 'Arun', 'Meena', 'Rahul', 'Kiran'}
2. Students who joined both activities: {'Priya', 'Kiran'}
3. Students who joined only sports: {'Rahul', 'Anil'}
4. Students who joined only music: {'Arun', 'Meena'}

Question 9: Consider the following string:

"Python Java C C++ Python Java Python"

Write a Python program to:

1. Convert the string into a list of programming languages.
2. Display the list.
3. Count how many times "Python" appears.

4. Remove duplicate language names.

5. Join the unique language names using " | ".

Answer :- Python

```
# Initial string
```

```
text = "Python Java C C++ Python Java Python"
```

```
# 1. Convert the string into a list of programming languages
```

```
languages_list = text.split()
```

```
# 2. Display the list
```

```
print("1 & 2. List of programming languages:")
```

```
print(languages_list)
```

```
print("-" * 50)
```

```
# 3. Count how many times "Python" appears
```

```
python_count = languages_list.count("Python")
```

```
print(f"3. 'Python' appears {python_count} times.")
```

```
print("-" * 50)
```

4. Remove duplicate language names (preserving order)

```
unique_languages = list(dict.fromkeys(languages_list))
```

```
print("4. Unique language names:")
```

```
print(unique_languages)
```

```
print("-" * 50)
```

5. Join the unique language names using " | "

```
joined_string = " | ".join(unique_languages)
```

```
print("5. Joined unique languages:")
```

```
print(joined_string)
```

Program Output

Plaintext

1 & 2. List of programming languages:

```
['Python', 'Java', 'C', 'C++', 'Python', 'Java', 'Python']
```

3. 'Python' appears 3 times.

4. Unique language names:

`['Python', 'Java', 'C', 'C++']`

5. Joined unique languages:

`Python | Java | C | C++`

Question 10: Write a Python program to print multiplication tables from 1 to 5.

Use nested loops to display each table from 1×1 up to 5×10 .

Also modify the program so that:

- The number 5 is skipped using `continue`.
- The inner loop stops when the multiplier reaches 8 using `break`

Answer :- Aap Question 10 ka code aur output direct copy-paste kar sakte ho:

Python Code Solution

Python

Part 1: Basic Multiplication Tables from 1 to 5

```
print("=== Standard Multiplication Tables (1 to 5) ===")
```

```
for i in range(1, 6):
```

```
    print(f"\n--- Table of {i} ---")
```

```
    for j in range(1, 11):
```

```
        print(f"{i} x {j} = {i * j}")
```

Part 2: Modified Program with 'continue' and 'break'

Requirement 1: Skip the multiplier 5 using 'continue'

Requirement 2: Stop inner loop when multiplier reaches 8 using 'break'

```
print("\n" + "=" * 45)
```

```
print("=== Modified Multiplication Tables ===")
```

```
for i in range(1, 6):  
  
    print(f"\n--- Table of {i} ---")  
  
    for j in range(1, 11):  
  
        # Stop the inner loop when multiplier reaches 8  
  
        if j == 8:  
  
            break  
  
  
        # Skip printing when multiplier is 5  
  
        if j == 5:  
  
            continue  
  
  
        print(f"{i} x {j} = {i * j}")
```

Program Output (Modified Version)

Plaintext

=== Modified Multiplication Tables ===

--- Table of 1 ---

$$1 \times 1 = 1$$

$$1 \times 2 = 2$$

$$1 \times 3 = 3$$

$$1 \times 4 = 4$$

$$1 \times 6 = 6$$

$$1 \times 7 = 7$$

--- Table of 2 ---

$$2 \times 1 = 2$$

$$2 \times 2 = 4$$

$$2 \times 3 = 6$$

$$2 \times 4 = 8$$

$$2 \times 6 = 12$$

$$2 \times 7 = 14$$

--- Table of 3 ---

$$3 \times 1 = 3$$

$$3 \times 2 = 6$$

$$3 \times 3 = 9$$

$$3 \times 4 = 12$$

$$3 \times 6 = 18$$

$$3 \times 7 = 21$$

--- Table of 4 ---

$$4 \times 1 = 4$$

$$4 \times 2 = 8$$

$$4 \times 3 = 12$$

$$4 \times 4 = 16$$

$$4 \times 6 = 24$$

$$4 \times 7 = 28$$

--- Table of 5 ---

$$5 \times 1 = 5$$

$$5 \times 2 = 10$$

$$5 \times 3 = 15$$

$$5 \times 4 = 20$$

$$5 \times 6 = 30$$

$$5 \times 7 = 35$$